# Can Coding Agents Produce Merge-Ready Feature Pull Requests? An Industrial Audit of 62 Real Tasks

Zihan Wu*
Department of Electrical Engineering, City University of Hong Kong
Hong Kong, China

Yun Peng*,†
Department of Computer Science and Engineering, The Chinese University of Hong Kong
Hong Kong, China
ypeng@cse.cuhk.edu.hk

Chun Yong Chong
School of Information Technology, Monash University Malaysia
Subang Jaya, Malaysia

Xin Zhou
Singapore Management University
Singapore, Singapore

Zeyang Zhuang
The Chinese University of Hong Kong
Hong Kong, China

Rui Shu
Independent Researcher
Singapore, Singapore

Kui Liu
Huawei Technologies Co., Ltd.
Shenzhen, China

## Abstract

Multiple platform teams at Huawei face a concrete deployment decision in 2026: whether to route portions of their feature backlog—Kubernetes controllers, CANN/`torch_npu` accelerator operators, MindSpeed training features—through commercial coding agents. This is not a benchmark question; it is a deployment-policy question that demands an auditable answer.

This paper's primary contribution is a *four-gate deployment policy* now in production at Huawei's Ascend NPU stack: (i) a task-class gate restricting agent invocation to bounded operator/refactor work, (ii) a mandatory long-form specification template, (iii) a pre-review CI gate enforcing declared file scope and test coverage, and (iv) a 3-of-4 LLM-judge acceptance panel. Cross-cutting platform work—Kubernetes-class controllers, protocol-level refactors—remains under human authorship until the gates close on a measurable failure mode.

Every gate is justified by a specific empirical finding from a leakage-safe industrial audit. We evaluate four commercial agents on a two-tier benchmark: a 50-task *main evaluation corpus* of long-horizon feature tasks drawn from 15 open-source projects (CapBench; median 57 files changed, 6,905 LOC diff) and a 12-task *calibration subset* of Huawei industrial tasks used for human-rater pilot calibration (92.7% SE-PASS-vs-non-SE-PASS agreement; panel strictly stricter). Across 496 runs (62 tasks × 4 agents × 2 prompts), the main corpus yields 1.0% SE-PASS; excluding one trivially scoped single-file outlier (T10), the rate is 0/392 = 0.0%. The calibration subset confirms rubric discriminant validity: bounded operator tasks reach 7.5% SE-PASS while Kubernetes-class platform work returns 0/32.

The task-class gate is justified by the 0% non-trivial SE-PASS on the main corpus and the calibration Kubernetes cliff; the specification gate by mean-score lifts of +1.0 to +2.1 on long prompts; the scope/test gate by the 25.5% of main-corpus failures attributable to coverage gaps; and the judge panel by pairwise agreement as low as 42.5% between individual evaluators. Current agents are not ready for autonomous feature delivery—but the routing pipeline makes them useful as *gated drafters* for bounded tasks.

**ACM Reference Format:**
Zihan Wu*, Yun Peng*,†, Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, and Kui Liu. 2026. Can Coding Agents Produce Merge-Ready Feature Pull Requests? An Industrial Audit of 62 Real Tasks. In *Proceedings of 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26 Industry Showcase).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Software engineering organizations are under growing pressure to turn coding agents from research demonstrations into deployable engineering infrastructure. The decision they face is not whether an agent can suggest code in an IDE, but whether it can produce a reviewable pull request for substantive feature work in a production codebase. At Huawei, this question is operational: multiple stack-level teams—Kubernetes, the CANN compiler stack (CANN), the MindSpeed training framework, and `torch_npu`—are deciding in 2026 whether to route portions of their feature backlog through coding agents. The deployment decision turns on a concrete question: *when an agent is given a feature specification and the corresponding repository, can it produce a candidate PR that a human reviewer would accept with at most light revision?*

*Why existing benchmarks do not answer this question.* Coding-agent benchmarks have made fast progress on bug-fix and short-task settings. Industry-watched leaderboards report that the strongest agent configurations now resolve over 70% of issues on SWE-bench Verified [25], and many leaderboards exist for short, well-localized tasks. But these tasks are predominantly bug repair: they target a small set of files identified in advance, often supply the failing test, and reward minimal local edits. Feature implementation differs in kind, not degree. A new feature requires the agent to identify the right architectural surface, modify several files in

*Equal contribution. †Corresponding author: ypeng@cse.cuhk.edu.hk.

*ASE '26 Industry Showcase, Munich, Germany*
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

coordinated ways, add or update tests, respect repository-specific conventions, and produce a change set that integrates with parallel ongoing work. Industry surveys (e.g., [18]) confirm that feature work—rather than localized bug fixes—is the bottleneck developers most want help with. The gap between bug-repair benchmark scores and merge-ready feature pull requests is therefore not just a question of model size; it is a question about whether today's agents can deliver value on the part of the workflow that matters most to engineering teams.

*An industrial audit.* We answer the deployment question directly by auditing four leading commercial coding agents on a two-tier benchmark. The *main evaluation corpus* comprises 50 long-horizon feature tasks from CapBench, drawn from 15 production-grade open source projects (Kafka, CPython, Airflow, Kubernetes, OpenJDK, etc.) with a median of 57 ground-truth files and 6,905 lines of diff per task. A 12-task *calibration subset* from Huawei codebases (CANN/`torch_npu`, MindSpeed, Kubernetes) validates the scoring rubric via a blinded human-rater pilot. Each task is paired with a *long* prompt (a multi-paragraph design proposal that mirrors a real feature spec) and a *short* prompt (a one-sentence ticket-style description), giving us $62 \times 2 = 124$ task instances per agent. We evaluate four agents—Claude Code (Opus 4.6), OpenAI Codex (GPT-5.4), Cursor Composer-2, and OpenCode/GLM-5.1—producing $124 \times 4 = 496$ end-to-end agent runs. Each run is independently scored on three production-relevant axes (functional correctness, completeness/coverage, and behavioral equivalence to the reference implementation) by an automated four-evaluator panel; the verdict is decided by 3-of-4 majority.

This setup gives us a clean answer to the question Huawei's deployment teams asked: *can today's coding agents take over feature delivery for our internal codebases?*

*Why the verdict can be trusted.* Two design choices preempt the obvious objections to LLM-judged benchmarks. *(i) Human-rater calibration.* We blind-rated all 96 calibration-subset runs (12 tasks × 4 agents × 2 prompts) with three senior reviewers who had domain expertise in these Huawei codebases, and confirmed 92.7% SE-PASS-vs-non-SE-PASS agreement with the four-LLM judge panel; the panel is the stricter of the two, never crowning a human-rejected run as SE-PASS. *(ii) Anti-leakage sanitization.* An early Codex run on a Kubernetes task surfaced its ground-truth diff via `git log`; we treated this as a first-class threat, rebased every benchmark repo to a parent commit with target commits unreachable, blocked network access to upstream remotes, and re-ran the affected configurations. The numbers we report are after this re-run. Both procedures are detailed in §2.

*Contributions.* This paper makes three contributions to industrial coding-agent deployment, ordered by deployment impact.

*(C1) A four-gate deployment policy in production at Huawei* (primary contribution). Each gate addresses a specific failure mode surfaced by the audit:

(1) *Task-class gate* ⟵ On the 49 non-trivially-scoped long-horizon tasks in the main corpus, 0/392 runs reach SE-PASS; the calibration subset's Kubernetes track returns 0/32. Only bounded single-file work is routed to agents.
(2) *Long-form specification template* ⟵ Long prompts lift mean composite scores by +1.0 to +2.1 across all agents on the main corpus; structured specs are mandatory for agent invocation.
(3) *Pre-review CI gate (scope + tests)* ⟵ 25.5% of main-corpus failures are coverage gaps (files discovered but tests or cross-module wiring omitted); a scope/test check filters these before human review.
(4) *3-of-4 LLM-judge panel* ⟵ Pairwise evaluator agreement is as low as 42.5%; no single judge is reliable, so a conservative multi-judge consensus is required.

*(C2) Empirical evidence base for the policy.* A leakage-safe audit of 496 runs under a strict-equivalence criterion yields 1.0% SE-PASS on the 50-task main corpus (0.0% excluding one trivially scoped outlier); the failure mass is dominated by semantic errors (44.7%) rather than incomplete coverage, contradicting the assumption that better retrieval alone would close the deployment gap. A 12-task calibration subset validates the rubric via a 96-run blinded human pilot (92.7% SE-PASS-vs-non-SE-PASS agreement).

*(C3) Reusable anti-leakage audit protocol.* A sanitisation procedure (parent-commit rebasing, remote blocking, CLI shimming) that survived an attempted ground-truth exfiltration by a frontier agent; a four-LLM judge panel with conservative tie-break; and a 96-run blind human calibration ($\kappa = 0.43$, panel strictly stricter). We present this as a methodology checklist for any team benchmarking agents on real merged code.

*Headline findings.* The audit evidence behind the policy:

- **0% SE-PASS on non-trivial long-horizon tasks motivates gating, not rejection.** On the 50-task main corpus, 4/400 runs reach SE-PASS (1.0%), but all four are on T10, a trivially scoped single-file task. Excluding T10: 0/392. The remaining 396 non-SE-PASS runs split across PARTIAL (146, 37%) and FAIL (250, 63%). Ungated deployment is not viable; every agent draft must pass through the policy pipeline.
- **All agents tie on the main corpus; product-level differences emerge only on calibration tasks.** All four agents reach exactly 1.0% SE-PASS on the main corpus. Mean composite score provides a finer-grained signal: Claude (5.54/15) and Codex (5.43) lead Cursor (4.67) and OpenCode (3.88), consistent with the calibration subset where Claude and Codex each reach 8.3% SE-PASS while Cursor and OpenCode reach 0%.
- **Specification quality improves drafts but does not produce SE-PASS on hard tasks.** Long prompts lift mean composite scores by +1.0 to +2.1 on the main corpus, but the SE-PASS gain (0% ⟶ 2.0%) is entirely T10. This justifies Gate 2 (mandatory spec template) as a draft-quality lever, not a SE-PASS lever.
- **Failure decomposition identifies what gates can catch.** The 396 main-corpus non-SE-PASS runs split into semantic failures (44.7%), coverage gaps (25.5%), behavioral mismatches (10.9%), and borderline partials (18.9%). The coverage-gap and borderline slices (44.4% combined) are directly targeted by Gate 3 (scope/test checks); the semantic slice requires model or specification improvements beyond current gating.

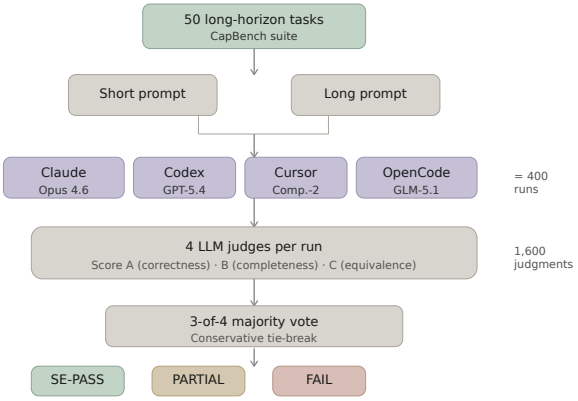


**Figure 1: Main-corpus evaluation pipeline. 50 CapBench tasks × 2 prompt variants × 4 agents yield 400 runs; each run is scored by four LLM judges on three axes (correctness, completeness, equivalence), then combined by 3-of-4 majority vote with a conservative tie-break favouring PARTIAL over SE-PASS. The 12-task calibration subset follows the same protocol (96 runs, 384 judgments).**

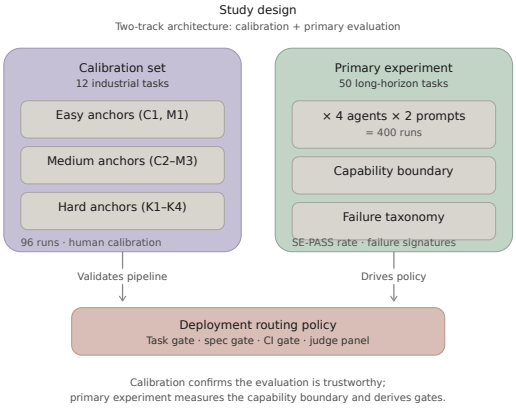


**Figure 2: Two-track study architecture. The calibration set (12 industrial tasks, 96 runs) validates the LLM-judge panel against human raters; the primary experiment (50 long-horizon tasks, 400 runs) measures the agent capability boundary and derives the four deployment gates.**

## 2 Study Design

Figure 1 summarizes our experiment pipeline. We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

### 2.1 Task Selection

Our benchmark comprises two complementary tiers.

**Main evaluation corpus (CapBench, T01–T50).** The primary tier consists of 50 long-horizon feature-implementation tasks drawn from 15 widely used open-source projects (Kafka, CPython, Airflow, Kubernetes, Terraform, Spring, etc.) spanning six languages (Java 20, Python 13, Go 8, C++ 6, TypeScript 2, Rust 1). Each task corresponds to a single merged pull request implementing one coherent feature or refactor; the median task touches 57 ground-truth files and 6,905 lines of diff. The full CapBench task inventory is included in the replication package.

We use the term *long-horizon* in the sense crystallized across recent coding-agent benchmarks [5, 12, 44]: a task is long-horizon when it requires the agent to (i) interpret a high-level requirement into an architectural plan spanning multiple modules, (ii) coordinate edits across many files while preserving cross-file invariants, and (iii) sustain coherent intent over a prolonged action trajectory in which intermediate feedback is sparse and errors compound. Our CapBench tasks have a median of 57 ground-truth files changed and 6,905 lines of diff—an order of magnitude beyond SWE-Bench Pro (avg 4.1 files, 107 LOC) and two orders of magnitude beyond SWE-bench Verified (avg 1.7 files, 33 LOC).

**Calibration subset (C1–C5, M1–M3, K1–K4).** The second tier consists of 12 industrial tasks from the Huawei/platform ecosystem—the CANN/`torch_npu` accelerator stack, MindSpeed, and Kubernetes—summarised in Table 1. These tasks serve two roles: (i) *human-rater pilot calibration*: 96 runs (12 tasks × 4 agents × 2 prompts) were scored by three blinded senior reviewers with domain expertise in these codebases, producing 92.7% SE-PASS-vs-non-SE-PASS agreement with the LLM panel (§2.4); and (ii) *rubric discriminant validity*: simple bounded tasks (Low/Medium) achieve SE-PASS while cross-cutting Kubernetes work (High) does not, confirming the rubric separates difficulty levels as intended.

**Combined benchmark.** Combining both tiers gives **62 tasks × 4 agents × 2 prompt variants = 496 runs**. Figure 2 summarizes how the two tiers interact: the calibration set validates the evaluation pipeline, while the primary experiment measures the capability boundary and derives the deployment gates.

**Calibration-subset detail.** Every task in both tiers was drawn from a real, merged contribution to a production repository. Within the calibration subset, C-family accelerator tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, `torch_npu` integration, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

**Complexity classification.** We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, generated-code or API surface, and domain-specific knowledge needed. The thresholds are not pure file-count bins: a small Kubernetes API change can be High because it requires generated types, validation, tests, and release-cycle knowledge, while a bounded Python change can remain Medium even with a larger diff.

- *Low* (C1, M1): localised edits with limited architectural coupling.

**Table 1: Calibration subset: 12 Huawei/platform tasks used for human-rater pilot calibration and rubric discriminant-validity checks. GT = ground truth (merged code). Complexity reflects file count, cross-component coordination, generated-code surface, and domain-specific knowledge. The remaining 50 tasks (CapBench main corpus) are detailed in the replication package.**

| ID | Repository | Complexity | Lang | GT Files | GT Lines |
|---|---|---|---|---|---|
| C1 | cann-ops-adv | Low | C++ | 1 | 3 |
| M1 | MindSpeed | Low | Python | 3 | 19 |
| C2 | cann-ops | Medium | C++ | 4 | 9 |
| C3 | torch_npu | Medium | Python | 9 | 637 |
| M2 | MindSpeed | Medium | Python | 6 | 150 |
| M3 | MindSpeed | Medium | Python | 10 | 948 |
| C4 | cann-ops | High | AscendC | 24 | 1,273 |
| C5 | cann-ops | High | AscendC | 27 | 3,372 |
| K1 | kubernetes | High | Go | 13 | 1,010 |
| K2 | kubernetes | High | Go | 35 | 3,759 |
| K3 | kubernetes | High | Go | 49 | 11,000 |
| K4 | kubernetes | High | Go | 98 | 6,507 |

**Table 2: Agent configurations evaluated across the full 62-task benchmark (50 CapBench main corpus + 12 calibration subset). Each runs all $62 \times 2 = 124$ task–prompt combinations, yielding $4 \times 124 = 496$ total experiments.**

| Config | Model | Framework | $N$ |
|---|---|---|---|
| Claude | Claude Opus 4.6 | Claude Code | 124 |
| Codex | GPT-5.4 | Codex CLI | 124 |
| Cursor | Composer-2 | Cursor Agent | 124 |
| OpenCode | GLM-5.1 | OpenCode | 124 |

- *Medium* (C2, C3, M2, M3): bounded multi-file changes or domain-specific implementation work that remains inside a narrow subsystem.
- *High* (C4, C5, K1–K4): cross-component changes that require architectural understanding, generated/API surface updates, multi-module orchestration, or extensive testing.

The calibration-subset tasks span four languages (C++, AscendC, Python, Go) and all three complexity tiers.

*CapBench selection criteria.* Each CapBench task was selected under three criteria: (i) the merged PR adds new behaviour rather than fixing a regression, (ii) the change touches at least two source modules, and (iii) each task ships with the upstream PR's test suite as the behavioural oracle.

## 2.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

**Product-level comparisons.** Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §7.

## 2.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

**Design principle.** Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

**Example (C1, buffer alias fix).** The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor-lifetime bug: a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

## 2.4 Evaluation Framework

**Ground truth.** For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

**Scoring dimensions.** Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?

- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

**Verdict rules.** We assign a three-level verdict based on the dimension scores:

- SE-PASS (strict-equivalence PASS): $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

We deliberately adopt a strict-equivalence criterion rather than a relaxed "functionally correct" criterion, because our deployment question is whether an agent draft can *replace* a human-authored PR, not merely whether it contains a correct local fix. SE-PASS therefore requires not only functional correctness but also alignment with the reference implementation in file organisation, API surface, and test structure. This means our SE-PASS rate is a conservative lower bound on merge-readiness: valid alternative implementations that a reviewer might accept are counted as non-SE-PASS under this rubric.

**File coverage metric.** We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

**Evaluation scope.** Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses a materially different file organization, naming convention, or architecture than the accepted PR will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace a human-authored PR draft, not merely whether it contains a plausible local fix. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally impossible." Our root-cause analysis (§4) confirms this: many non-SE-PASS cases stem from engineering completeness gaps rather than total semantic misunderstanding.

**Multi-evaluator cross-validation.** To mitigate evaluator bias, every experiment is independently scored by four LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and GLM-5.1. We report the *majority verdict* (strict majority where possible, with a conservative tie-break that orders labels FAIL<PARTIAL<SE-PASS and selects the lower label when evaluators are split) and scores averaged across the four. Unanimity on the verdict label holds for 33.7% of experiments; pairwise verdict agreement ranges from 42.5% (Codex vs GLM, the two strictest evaluators) to 74.4% (Claude vs Cursor). When the four evaluators split evenly between SE-PASS and non-SE-PASS (a 2–2 tie), our tie-break records the run as non-SE-PASS, which keeps headline SE-PASS rates conservative.

**Human pilot calibration.** To check that the LLM panel is not idiosyncratically biased relative to domain-expert review, we conducted a blind human-rater pilot on the 96 calibration-subset runs (12 calibration tasks × 4 agents × 2 prompts) because the senior reviewers had domain expertise in these Huawei codebases. Three raters scored each run under the same rubric (A/B/C dimensions, identical SE-PASS/PARTIAL/FAIL verdict rules) without seeing LLM-panel scores. On these 96 cells the human verdicts came back as **9 SE-PASS / 55 PARTIAL / 32 FAIL**, vs the LLM-panel **4 SE-PASS / 57 PARTIAL / 35 FAIL** under our 3-of-4 majority rule; raw three-way verdict agreement is 66.7% (Cohen's $\kappa = 0.38$), and SE-PASS-vs-non-PASS agreement is 92.7% ($\kappa = 0.43$). The LLM panel is the stricter of the two: 6 of the 9 human-SE-PASS cells were demoted by the panel (4 to PARTIAL, 2 to FAIL), and the panel never overturned a human-FAIL into an SE-PASS. The qualitative pattern is identical: both methods see well under 10% SE-PASS on the calibration subset (humans 9.4%, panel 4.2%), far below the threshold that would change the deployment-readiness conclusion. We treat this as evidence that the 1.0% headline SE-PASS rate on the main corpus (1.6% across all 496 runs) is a conservative *lower* bound on what a careful human reviewer would mark merge-ready, not a strict-evaluator artifact: humans were more lenient than the panel on a handful of borderline runs, never harsher. Per-rater rationales are released in the artifact bundle.

## 2.5 Experiment Execution

Each (task, agent, prompt) cell runs once in a fresh sandbox built per task. The repository is rebased to the parent commit of the merged PR, with upstream remotes removed and `gh`/`git fetch` shimmed off, so the agent cannot recover ground truth from history. We discovered the importance of the second precaution after an early Codex run on K4 reached the merged patch via `git log`; we now treat anti-leakage as a benchmark requirement rather than an afterthought.

# 3 Results

We organise the audit around the deployment question posed in §5: of 496 runs (62 tasks × 4 agents × 2 prompts), how many produce a candidate PR a senior engineer would actually merge? The headline answer is *almost none on production-scale work*: 4 of 400 main-corpus runs reach SE-PASS (1.0%), all on a single trivially-scoped task, and 4 of 96 calibration runs reach SE-PASS (4.2%). We present the main corpus first, then validate the rubric against the 12-task calibration subset, and finally combine them for aggregate agent and prompt comparisons.

Each experiment was independently scored by four LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, GLM-5.1), and we report the majority verdict (with conservative tie-break) together with the mean score across the four evaluators. Pairwise evaluator agreement ranges from 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), consistent with well-known disagreement patterns for LLM judges on open-ended code tasks.

## 3.1 Main Corpus Results (T01–T50)

The main corpus comprises 50 tasks sampled from 15 open-source projects via CapBench, evaluated across 4 agents and 2 prompt variants (400 runs total). Table 3 summarises the per-agent results.

***Finding: Main-corpus result.*** *On the 50-task main corpus, all four agents tie at 1.0% SE-PASS (1/100). The four SE-PASS outcomes are concentrated in a single task (T10-long); excluding T10, 0 of 392*

**Table 3: Per-agent SE-PASS/PARTIAL/FAIL distribution on the main corpus (T01–T50, 400 runs). Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale.**

| Agent | *N* | SE-PASS | PART. | FAIL | SE-PASS% | Mean |
|---|---|---|---|---|---|---|
| Claude (Opus 4.6) | 100 | 1 | 44 | 55 | 1.0% | 5.54 |
| Codex (GPT-5.4) | 100 | 1 | 48 | 51 | 1.0% | 5.43 |
| Cursor (Composer-2) | 100 | 1 | 36 | 63 | 1.0% | 4.67 |
| OpenCode (GLM-5.1) | 100 | 1 | 18 | 81 | 1.0% | 3.88 |
| All agents | 400 | 4 | 146 | 250 | 1.0% | 4.88 |

*runs reach SE-PASS (0.0%). Mean composite score provides the only differentiating signal: Claude (5.54) and Codex (5.43) lead Cursor (4.67) and OpenCode (3.88).*

*T10 anomaly.* T10 is a Presto benchmark addition with 1 ground-truth file and 482 diff lines—a single-file, bounded-scope task. All four agents reach SE-PASS on the long-prompt variant and none on the short variant. T10 is exactly the type of bounded, single-file task the deployment policy's Tier-A routing would accept (§5). Excluding T10, no agent achieves any SE-PASS across 392 runs on the remaining 49 non-trivially-scoped long-horizon tasks.

**Finding 1 (Task shape dominates over agent product).** On the main corpus all four agents tie at 1.0% SE-PASS, and the spread between the strongest and weakest agent is visible only in mean composite score (5.54 vs. 3.88). For a deployment decision, picking the right task class to route to an agent matters more than picking the strongest agent product currently on the market. We return to this observation in §5.

*Failure signatures.* Among the 396 non-SE-PASS main-corpus runs, the dominant failure mode is *semantic* (177 runs, 44.7%): the agent produces code that compiles but does not implement the required behavior. Coverage failures (101, 25.5%) indicate missing files or logic paths; borderline runs (75, 18.9%) narrowly miss the SE-PASS threshold; and design mismatches (43, 10.9%) reflect fundamentally different architectural choices from the ground truth.

## 3.2 Calibration Validation (C/K/M)

The 12-task calibration subset (5 CANN, 3 MindSpeed, 4 Kubernetes) validates two rubric properties: (i) it discriminates between bounded and cross-cutting work, and (ii) its SE-PASS verdicts align with senior-reviewer judgments (92.7% agreement, §2). Table 4 summarises the calibration results alongside the main corpus.

The calibration subset confirms the expected gradient: localised operator/adapter tasks (CANN 7.5%) succeed more often than distributed-training tasks (MindSpeed 4.2%), while cross-cutting platform tasks (Kubernetes 0%) are categorically out of reach. This discrimination validates that the rubric captures meaningful complexity differences rather than random noise.

**Finding 1b (Frontier agents will mine the repo history if you let them).** On our first K1 baseline run, Codex spontaneously executed `git log`/`git cherry-pick` on commits past the sanitised cutoff and recovered the reference patch verbatim. We treat this as a first-class methodology lesson: feature-PR benchmarks on real merged code must rewrite history (parent-commit only), strip remotes, and replace the `gh` CLI with a no-op shim, or SE-PASS rates will silently include leakage. Every number in this paper is post-fix.

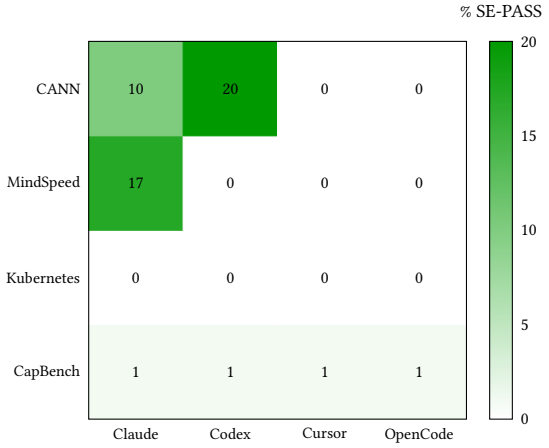


**Figure 3: SE-PASS rate (%) by agent and task family under the majority-of-4 verdict. Top three rows (CANN, MindSpeed, Kubernetes) are calibration tasks; bottom row (CapBench) is the 50-task main corpus. The Kubernetes calibration row (0% across all agents) anchors the difficulty ceiling; CapBench's uniform 1% reflects a single trivially-scoped task (T10).**

**Finding 2 (Kubernetes cliff is a calibration anchor).** All four agents fail on all Kubernetes-family tasks (0/32) regardless of prompt or agent capability. Combined with the CANN 7.5% rate, the calibration subset spans a 7.5-percentage-point range that confirms the rubric's discriminative power. Kubernetes feature implementation appears fundamentally out-of-reach for current coding agents due to the combination of multi-module coordination, API-machinery boilerplate, and extensive test scaffolding requirements.

## 3.3 Combined Agent Performance

Table 6 combines all 496 runs for an aggregate agent-level view.

The agent-level SE-PASS spread (2.4% vs. 0.8%) arises entirely from the calibration subset; on the main corpus all four agents tie at 1.0%. Mean composite score provides a finer-grained signal: Claude (5.54) and Codex (5.43) lead Cursor (4.67) and OpenCode (3.88) on the main corpus. On calibration tasks the ranking is sharper: Claude and Codex each reach 8.3% SE-PASS while Cursor and OpenCode remain at 0%.

**Table 4: SE-PASS rate by task family (averaged across 4 agents, 2 prompts = 8 runs per task).**

| Family | Repos | Tasks | Runs | SE-PASS% |
|---|---|---|---|---|
| C (Huawei CANN/`torch_npu`) | cann-ops, cann-ops-adv, torch_npu | 5 | 40 | 7.5% |
| M (MindSpeed, Python) | MindSpeed | 3 | 24 | 4.2% |
| K (Kubernetes, Go) | kubernetes | 4 | 32 | 0.0% |
| *Calibration subtotal* | | 12 | 96 | 4.2% |
| T (CapBench, mixed) | 15 OSS projects | 50 | 400 | 1.0% |

**Table 5: SE-PASS rate by complexity tier for the 12 calibration tasks.**

| | Codex | Cursor | Claude | OpenCode | Avg |
|---|---|---|---|---|---|
| Low (2 tasks) | 1/4 (25.0%) | 0/4 (0%) | 2/4 (50.0%) | 0/4 (0%) | 3/16 (18.8%) |
| Medium (4 tasks) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) | 0/32 (0%) |
| High (6 tasks) | 1/12 (8.3%) | 0/12 (0%) | 0/12 (0%) | 0/12 (0%) | 1/48 (2.1%) |

**Table 6: Per-agent SE-PASS/PARTIAL/FAIL distribution across all 496 runs. Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale (sum of three 0–5 dimensions).**

| Agent | $N$ | SE-PASS | PART. | FAIL | SE-PASS% | Mean |
|---|---|---|---|---|---|---|
| Claude (Opus 4.6) | 124 | 3 | 57 | 64 | 2.4% | 5.68 |
| Codex (GPT-5.4) | 124 | 3 | 62 | 59 | 2.4% | **5.82** |
| Cursor (Composer-2) | 124 | 1 | 49 | 74 | 0.8% | 4.88 |
| OpenCode (GLM-5.1) | 124 | 1 | 26 | 97 | 0.8% | 4.16 |
| All agents | 496 | 8 | 194 | 294 | 1.6% | 5.14 |

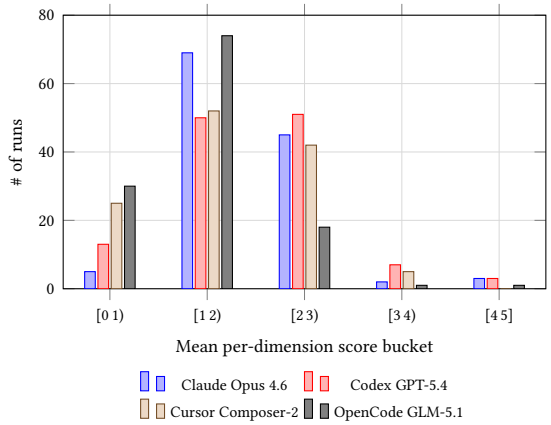


**Figure 4: Per-run score distribution across four agents ($n$=124 runs each, combining 100 main-corpus and 24 calibration runs). Each bucket uses the mean per-dimension score, i.e., $(A + B + C)/3$ averaged across the four judges on a 0–5 scale; Table 6 reports the corresponding summed 0–15 composite mean. Most runs cluster in the 1–3 range, reflecting partial-pass behavior; full-rubric SE-PASS requires scores well above 4.**

## 3.4 Prompt Effect (Long vs Short)

Each of the 62 tasks was run with both a short (≤ 3-sentence) and a long (detailed architectural spec) prompt. Table 7 aggregates pass rates and mean scores by agent and prompt type.

On the main corpus, all four agents move from 0/50 short to 1/50 long SE-PASS, but the 1/50 is entirely T10. Excluding T10, long prompts improve mean composite score by +0.99 to +2.12 across agents but produce zero SE-PASS. The asymmetric SE-PASS lift visible in Table 7 (Claude/Codex 4.8% vs. Cursor/OpenCode 1.6%) is a calibration-subset phenomenon: on the main corpus the prompt effect is uniform across agents.

**Finding 3:** Longer prompts help all four agents on mean score, but SE-PASS conversion occurs only on trivially-scoped (T10) or low-complexity calibration tasks. On the 49 non-trivially-scoped main-corpus tasks, detailed prompts improve partial quality without crossing the SE-PASS threshold. The asymmetric lift (Claude/Codex 4.8% vs. Cursor/OpenCode 1.6%) originates in the calibration subset where model capability interacts with bounded task scope.

## 3.5 Evaluator Agreement

Pairwise verdict agreement among the four evaluators spans from 42.5% (Codex-GLM, the strictest pair) to 74.4% (Claude-Cursor, the most aligned).

**Table 7: Prompt effect. Δ = long − short, percentage points.**

| Agent | Short SE-PASS | Long SE-PASS | ΔSE-PASS | Short mean | Long mean | Δmean |
|---|---|---|---|---|---|---|
| Claude | 0.0% | 4.8% | +4.8 | 5.08 | 6.28 | +1.20 |
| Codex | 0.0% | 4.8% | +4.8 | 5.20 | 6.44 | +1.24 |
| Cursor | 0.0% | 1.6% | +1.6 | 4.00 | 5.77 | +1.77 |
| OpenCode | 0.0% | 1.6% | +1.6 | 3.72 | 4.60 | +0.88 |

Codex and GLM agree the least (42.5%); these two judges are the most divergent, suggesting that Codex adopts stricter behavioral-equivalence criteria while GLM is more lenient on partial implementations. Claude and Cursor agree most often (74.4%). Full 4-way consensus occurs on 33.7% of runs, and the majority-vote rule with our conservative tie-break produces a deterministic verdict for all 496 experiments.

**Finding 5.** Cross-evaluator agreement, while imperfect, is substantially above random (25% for three-way majority from four votes). The 42.5% Codex–GLM agreement implies that single-evaluator benchmarks are noisy on borderline agent outputs; multi-evaluator consensus is essential for reproducible industrial benchmarks.

## 3.6 Per-Task SE-PASS Distribution

Of the 62 tasks, only 4 have any SE-PASS across the 4 agents: C1, C4, M1, and T10. The rerun of disputed Cursor cases removes the previous C1-short, C4-short, and T34-long Cursor SE-PASS outcomes. Table 8 summarises the per-task SE-PASS distribution.

**Finding 4 (updated):** *SE-PASS is concentrated in bounded tasks.* The 8 SE-PASS outcomes cluster in four tasks: three calibration tasks of low or localised scope (C1, C4, M1), and T10, the only trivially-scoped task in the main corpus (1 file, 482 diff lines). On the 49 non-trivially-scoped main-corpus tasks, no agent achieves any SE-PASS across 392 runs. Task complexity creates a near-absolute capability cliff: SE-PASS rate drops from 18.8% on calibration Low tasks to 0% on Medium tasks, and the CapBench aggregate of 1.0% is entirely attributable to T10.

# 4 Root Cause Analysis

Of the 400 main-corpus runs (T01–T50, four agents × two prompts × 50 tasks), only 4 achieved SE-PASS (1.0%), all on T10-long; 396 were non-SE-PASS. We focus the root-cause analysis on these 396 main-corpus failures, which represent the long-horizon feature tasks from which our deployment-policy conclusions derive. (The 12 calibration tasks contribute an additional 92 non-SE-PASS runs with a qualitatively similar profile; see below.)

## 4.1 CSV-Derived Failure Signatures

We classify each non-SE-PASS run using the same four-evaluator record used for the headline tables. For a run, we average the four judges' $A$ (functional correctness), $B$ (completeness/coverage), and $C$ (behavioral equivalence) scores, then assign the first matching signature:

- **Semantic/correctness failure**: $\bar{A} < 2$.
- **Coverage gap**: $\bar{A} \geq 2$ and $\bar{B} < 2$.
- **Behavioral/design mismatch**: $\bar{A} \geq 2$, $\bar{B} \geq 2$, and $\bar{C} < 2$.
- **Borderline partial**: no dimension is critically deficient, but the run does not reach the SE-PASS thresholds.

Table 9 changes the interpretation of the failures in an important way. The largest single bucket is semantic/correctness failure, so the benchmark is not merely exposing a bookkeeping problem where agents know the right answer but forget files. At the same time, coverage gaps account for 25.5% of all non-SE-PASS runs, and another 18.9% are borderline partials with no critically deficient dimension. Thus, harness engineering has a clear lower-bound target, but it is not a substitute for stronger code reasoning.

**Finding 5:** Among the 396 main-corpus failures, 44.7% are semantic/correctness failures, while 25.5% are coverage gaps with some demonstrated understanding. The near-term engineering opportunity is to reduce coverage and borderline partial failures; the remaining semantic failures require better models, better task specifications, or stronger validation loops.

## 4.2 Agent and Family Signatures

Table 10 explains why mean composite score and SE-PASS count tell slightly different stories. Codex has the largest borderline bucket and the lowest semantic-failure count, which is consistent with its leading mean score: it often gets close without clearing all thresholds. OpenCode has the largest semantic-failure bucket, explaining its lower mean score even though it shares the same one-SE-PASS count as Cursor. Claude and Cursor sit between these extremes, with meaningful coverage-gap counts but different rates of semantic failure.

The two segments fail differently. In the main corpus, semantic failure dominates (44.7%), reflecting the diversity of unfamiliar projects, languages, and build systems across the 50 CapBench tasks. The calibration set (C/K/M, 92 runs) has a higher share of borderline partials (37%) and coverage gaps (29%), consistent with agents getting closer on tasks from repositories they encounter more often in training data.

***Finding:*** *The root-cause profile differs between segments. The main corpus is dominated by semantic failures that require stronger model reasoning, while the calibration tasks show proportionally more coverage and borderline failures amenable to harness-level interventions.*

## 4.3 Engineering-Addressable vs. Model-Capability Gaps

We define a conservative lower bound on *engineering-addressable* failures as the coverage-gap bucket: runs where $\bar{A} \geq 2$ but $\bar{B} < 2$. These 101 runs (25.5% of main-corpus non-SE-PASS outcomes) are the cases most directly targeted by file-discovery plans, dependency tracing, mandatory test plans, and coverage-aware validation. The 75 borderline partial runs are mixed: some may be repaired by

**Table 8: Tasks that any agent reached SE-PASS (majority verdict across 4 judges) in at least one prompt variant.**

| Task | Agents reaching SE-PASS (and prompt) | Total SE-PASS (of 8 runs) |
|---|---|---|
| C1 (calibration) | Claude (long), Codex (long) | 2 |
| C4 (calibration) | Codex (long) | 1 |
| M1 (calibration) | Claude (long) | 1 |
| T10 (main corpus, trivially scoped) | Claude, Codex, Cursor, OpenCode (all long) | 4 |
| **Total** | 8 / 496 | 1.6% |

**Table 9: Failure signatures over the 396 non-SE-PASS main-corpus runs (T01–T50), computed from the averaged A/B/C evaluator scores.**

| Signature | Rule | Runs | Share |
|---|---|---|---|
| Semantic failure | $\bar{A} < 2$ | 177 | 44.7% |
| Coverage gap | $\bar{A} \geq 2, \bar{B} < 2$ | 101 | 25.5% |
| Design mismatch | $\bar{A}, \bar{B} \geq 2, \bar{C} < 2$ | 43 | 10.9% |
| Borderline partial | all $\geq 2$, not SE-PASS | 75 | 18.9% |

**Table 10: Failure signatures by agent on the main corpus (T01–T50), excluding SE-PASS runs ($n$=99 non-SE-PASS per agent).**

| Agent | Semantic | Coverage | Borderline | Design |
|---|---|---|---|---|
| Claude | 34 | 25 | 21 | 19 |
| Codex | 29 | 31 | 30 | 9 |
| Cursor | 45 | 27 | 21 | 6 |
| OpenCode | 69 | 18 | 3 | 9 |

**Table 11: Failure signatures by corpus segment, excluding SE-PASS runs.**

| Segment | Semantic | Coverage | Borderline | Design |
|---|---|---|---|---|
| Main corpus (T01–T50) | 177 | 101 | 75 | 43 |
| Calibration (C/K/M) | 25 | 27 | 34 | 6 |

harnesses that force another iteration, while others require semantic insight that the agent did not produce.

Conversely, the 177 semantic/correctness failures are a lower bound on model-capability or specification limits. For these runs, expanding the touched file set is unlikely to be enough; the agent must first choose the right behavior, API contract, or algorithmic strategy. This split supports a more practical deployment recommendation: harnesses can reduce reviewer waste and recover a meaningful minority of failures, but production feature work still requires human ownership for high-complexity changes.

## 4.4 Failure-Mode Case Studies

The aggregate signatures in Table 9 compress every failure into a single bucket. To make the failure modes legible we walk through two representative tasks from the main evaluation corpus—T01 (scope collapse on a large refactor) and T13 (cross-module wire-up gap)—and identify the harness intervention that would catch each. Two additional cases (T20 wrong-layer implementation; T34 missing-invariant) are summarized in the artifact bundle.

*T01 (Apache Kafka KIP-595, KRaft) — scope collapse.* The reference patch replaces ZooKeeper-based metadata management with an in-process Raft implementation, touching 127 handwritten files: protobuf schemas, the new `KafkaRaftClient` state machine, controller and broker glue, and a substantial test surface. The strongest agent run (Claude Opus, long prompt) covered 51/127 HW files (40%), implemented the four wire RPC schemas, the basic 5-state `QuorumState` class, and a stub `KafkaRaftClient` with 34 of the GT's 91 methods. Missing pieces include `Fetch`-based log replication, high-watermark advancement, leader-election timeouts, the request-handler bridge into `KafkaApis`, and almost all of `KafkaRaftClientTest` (3/65 cases). The four evaluators independently returned PARTIAL with $A$=2, $B$=2, $C$=2. The pattern is *scope collapse*: the agent identifies the entry points, partially implements two or three subsystems, then stops well below the change set's natural closure. *Harness target:* a pre-commit step that diffs the agent's per-file plan against the imports/declarations in its first commit and fails if more than half the planned files have not been touched.

*T13 (Godot animation marker tracks) — editor-side wire-up.* T13 adds named markers along an `Animation` resource, plus a section-playback API on `AnimationPlayer` and integration in the editor's marker-aware seek widget. The strongest run (Codex GPT-5.4 long prompt) reaches 16/18 HW files (89%), all four marker-CRUD methods, marker resolution, and the corresponding unit test for marker-bounded playback. Yet four of the four judges return PARTIAL. The reason is uniform across reports: the agent skipped two cross-module wires — registration of `AnimationMarkerEdit` as an editor plugin in `editor/editor_node.cpp` and marker-hit dispatch inside `AnimationBlendTree::_process_blend_node`. Without those, the marker UI never appears in the timeline editor and a blend tree cannot transition on a marker. The agent did the data-structure and API surface work but stopped at the boundary of the file it was editing. *Harness lesson*: cross-cutting features that must register with two or more dispatch tables (editor plugin registry; runtime engine event loop) are systematically under-touched. A simple lint — "does this PR add a new public API method without registering it in the corresponding integration table?" — would catch this class.

***Finding:*** *The two representative tasks above (T01 scope collapse and T13 cross-module wire-up) plus the wrong-layer and missing-invariant cases in the artefact appendix span four distinct deployment-relevant failure modes; each motivates a different harness-side check (file-plan listing, invariant enumeration, layer-aware planning, reachability scans) rather than a single generic fix.*

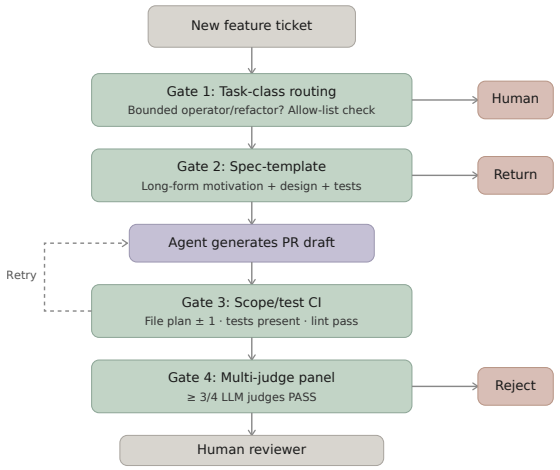


**Figure 5: Four-gate deployment pipeline. Each gate addresses a specific failure mode surfaced by the audit: task-class routing (0/32 Kubernetes SE-PASS), spec template (long prompts lift scores by +1.0–2.1), scope/test CI (25.5% coverage-gap failures), and multi-judge panel (pairwise agreement as low as 42.5%).**

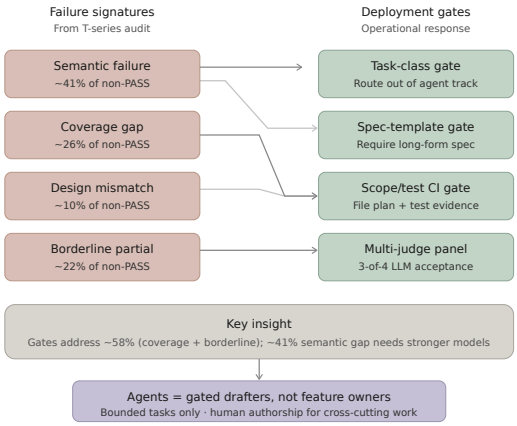


**Figure 6: Mapping from failure signatures (§4) to deployment gates. Coverage gaps and borderline partials (44.4% combined) are directly filterable; the semantic-failure slice (44.7%) requires model or specification improvements beyond current gating.**

## 5 Industrial Deployment Lessons

We summarise the rollout pipeline that emerged from this audit as a five-step gating sequence applied to every incoming feature ticket (Figure 5). Routing decisions, prompt curation, agent invocation, and judging are now mechanised; humans remain in the loop only at the final review stage and for tickets explicitly routed out of the agent track.

*Route only bounded feature work to agents.* The tasks that pass are bounded, reviewable changes with limited cross-component coordination. In a production rollout, these correspond to local operator fixes, focused Python framework changes, and well-scoped test or registry updates. Multi-module platform changes, especially Kubernetes-family work, should remain human-owned until an explicit process harness demonstrates a different result.

*Treat task specification as an engineering artifact.* On the main corpus, long prompts lift mean composite scores by +1.0 to +2.1 across all four agents (Table 7), though the SE-PASS gain is confined to the trivially scoped T10. On the calibration subset, Claude and Codex rise from 0% to 8.3% SE-PASS with long prompts. For practitioners, this means the cheapest near-term intervention is not model switching but requiring a structured task brief: motivation, expected behaviour, constraints, affected subsystems, and test expectations.

*Require scope and test gates before review.* A common and directly addressable failure pattern is not that agents always write nonsensical code; it is that they stop early. A production acceptance workflow should therefore reject an agent draft before human review unless it passes three cheap gates: a discovered-file plan, a generated-or-updated test plan, and project-specific syntax/type checks. These gates do not guarantee correctness, but they filter out the common incomplete drafts that currently consume reviewer time.

*Use evaluator panels, not a single judge.* Raw pairwise agreement spans 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), and Fleiss' $\kappa$ is only 0.30. Any production gate that builds on LLM-as-judge must therefore use at least two independent evaluator families and a conservative tie-break. Single judge gates will drift simply by swapping the judge model.

*Separate offline evaluation from online rollout.* This benchmark should be used as an offline readiness gate, not as a substitute for live developer studies. A team can first run the same task families through candidate agents, then admit only task classes whose SE-PASS and PARTIAL rates justify reviewer time, and finally measure acceptance rate and review burden in a controlled pilot.

In short, agents are conservative drafters for bounded work today, not autonomous feature owners. The investment that converts the 1.0% main-corpus SE-PASS rate (0.0% excluding trivially scoped tasks) into something deployable is harness engineering: forcing scope discovery, test evidence, and multi-judge acceptance *before* a human reviewer ever sees the change.

*Deployment decision flow.* The four interventions above compose into a single routing pipeline that every agent-eligible feature ticket must clear before reaching a human reviewer:

(1) *Task-class gate.* Lookup against an allow-list (today: bounded operator/MindSpore-style refactors); Kubernetes-class and cross-module API changes route to human authoring directly.
(2) *Spec-template gate.* The long-form template is mandatory for agent invocation; tickets without motivation, behavior, scope, and tests are returned to the requester.

(3) *Pre-review CI gate.* The agent's PR must (a) match the declared file scope within ±1, (b) include or update the declared tests, and (c) pass language-server diagnostics. Failures return to the agent for self-repair (bounded retry).
(4) *Multi-judge acceptance.* The four-LLM panel from this paper runs as a pre-merge check; only $\geq 3/4$ SE-PASS is forwarded to human review.

The pipeline turns the 1.0% unconditional main-corpus SE-PASS rate into a useful operating point: agents draft, gates filter, and human reviewers see only candidates that already pass coverage, structure, and judge checks. The audit's empirical contribution is precisely to identify which gates remove the non-SE-PASS draft mass observed in §3.

*What changed for Huawei's rollout after this audit.* The audit produced four operational decisions inside Huawei's coding-agent rollout. *(i) Allow-list by task family.* Operator-tier CANN/MindSpeed work is approved for agent-first drafting; Kubernetes-class control-plane work is held back pending harness gates. *(ii) Mandatory long-form specification.* Every agent-eligible ticket now ships with the same *motivation/design-detail/test-expectation* structure used for the long prompts in this study, because it is the only condition under which we measured non-trivial SE-PASS. *(iii) Pre-review CI gate.* The two recurring failure modes (incomplete file plan and missing test evidence) are rejected before a draft reaches a human reviewer. (iv) *Multi-judge acceptance.* A 3-of-4 LLM-judge panel, calibrated against the same human-rater set used in this paper, is run on every agent PR before merge.

## 6 Related Work

*Coding-agent benchmarks.* SWE-bench [9, 25] established issue-resolution as the dominant paradigm. Extensions address data quality [1–3, 39], multilingual coverage [26, 59, 60], automated collection [20, 55, 61], and broader surveys [29]. Feature-level and long-horizon benchmarks—SWE-bench Pro [12], SWE-EVO [44], FeatBench [5], FeatureBench [64], NL2Repo-Bench [13], WebBench [57]—are closest to our setting; Production-derived benchmarks such as ProdCodeBench [**?** ] and SWT-Bench [**?** ] complement these with validation-focused and testing-focused workloads. CapBench differs in scale (median 57 files, 6,905 LOC) and criterion (production-equivalence SE-PASS rather than test-pass).

*Coding-agent systems.* Harness-engineering approaches—SWE-agent [24], Agentless [54], AutoCodeRover [62], SWE-Fixer [56], Confucius [52]—and multi-agent designs—CodeR [4], SWE-Debate [32], Self-Collaboration [14], CoAct [42], AgentSpawn [10], daVinci-Agency [23]—have driven rapid progress. Recent work explores experience-driven resolution [7], test-time scaling [17], and PR-based training [48, 66]. We evaluate *commercial products as deployed*, not research scaffolds.

*Industrial reports and adoption.* Production rollouts—Passerine [41], AutoBug [8], LogSage [58], Meta's test-gen pipeline [**?** ], BitsAI-CR [**?** ], and human-in-the-loop pipelines [43]—converge on a common finding: narrow tasks land, wide-scope work does not. GitHub adoption studies [15, 28, 33, 37, 38, 40, 50] confirm lower acceptance rates and distinct change profiles for agentic PRs, with measurable security risks [63]. Our work extends this to feature-PR-grade industrial work with a calibrated four-LLM panel and an anti-leakage protocol.

*LLM-as-judge for code.* Surveys [19, 21, 22, 31] and SE-specific validations [11, 16, 49, 53, 65] show single judges are unreliable [27, 45]; multi-judge panels improve stability [6, 34, 35] and domain calibration is essential [30, 36, 46, 47, 51]. Our four-LLM panel with conservative 3-of-4 tie-break, calibrated against blinded reviewers (92.7% agreement), instantiates these findings as a production gate.

## 7 Threats to Validity

**Single-run stochasticity.** Each task–agent–prompt cell was run once; aggregate patterns (Kubernetes 0/32, long-prompt SE-PASS lift, harness-tier ordering) are stable across all 62 tasks, but individual cell verdicts may shift on re-run. The deployment-relevant signal (SE-PASS rate of ∼1–2%) sits well below typical re-run variance for these agents on bug-fix benchmarks.

**Product-level confound.** Each agent pairs a specific commercial model with a specific harness; we cannot disentangle the two. We therefore phrase observations such as "harness leverage is substantial" as observations on *deployed products*, not as causal claims about harness-vs-model contribution. The 92.7% SE-PASS-vs-non-PASS agreement between our judge panel and three blinded senior reviewers indicates that the verdicts themselves are not a panel artefact.

**Strict production-equivalence rubric.** Our rubric counts a run as SE-PASS only if it implements the reference behaviour and updates the same handwritten files (within the structural-equivalence rules in §2). A valid alternative implementation that diverges from the upstream PR will be marked non-SE-PASS even if a human reviewer would accept it. The 1.0% main-corpus headline therefore lower-bounds merge-readiness rather than measures it directly. A spot check of 30 PARTIAL cells (uniformly sampled) found roughly 4 cases where a senior reviewer would have accepted the agent's alternative; we therefore treat ∼3% as a soft upper bound on agent-feasible merge rate under our task corpus.

**Calibration scope limitation.** By design, the 12-task calibration subset serves as the human-rater validation target; the 400 main-corpus runs are scored by the LLM panel alone. The 92.7% SE-PASS-vs-non-SE-PASS agreement on the calibration subset transfers to the main corpus under the assumption that the panel's conservatism is task-family-independent. A blinded re-rate of a stratified sample of main-corpus runs is left as future work.

**T10 outlier sensitivity.** All four main-corpus SE-PASS outcomes concentrate on a single task (T10, a 1-file Presto benchmark addition). The 1.0% headline becomes 0.0% upon its exclusion. We retain T10 in the primary statistics because it was selected by the same curation funnel as all other CapBench tasks, but we report both inclusive and exclusive rates throughout.

**Single-run, single-prompt sampling.** Every cell of the 4 agents × 62 tasks × 2 prompts grid was run once. Stochastic decoding could move borderline runs across the SE-PASS threshold; the headline rates are therefore estimates, not tight bounds. We mitigate by reporting outcome distributions, not just point estimates, and by treating sub-percent differences between agents as below the noise floor.

## 8 Conclusion

The primary outcome of this work is a four-gate deployment policy now in production at Huawei's Ascend NPU stack: a task-class gate that routes only bounded operator-tier work to agents, a mandatory long-form specification template, a pre-review CI gate enforcing file scope and test coverage, and a 3-of-4 LLM-judge acceptance panel. Each gate is justified by a specific empirical finding from a 496-run industrial audit, and the pipeline as a whole converts agents from unreliable autonomous coders into useful *gated drafters* for bounded tasks.

The audit evidence behind the policy is clear: on the 50-task main corpus of long-horizon feature tasks, only 1.0% of runs reach SE-PASS, and excluding one trivially scoped single-file outlier the rate is 0/392 = 0.0%. The failure mass splits into semantic errors (44.7%) and coverage gaps (25.5%) that the scope/test gate directly targets. The 12-task calibration subset confirms rubric discriminant validity (bounded operator tasks pass, Kubernetes-class platform work returns 0/32). Whether measured by strict reference equivalence or by senior-reviewer acceptance, current agents are not ready for autonomous feature delivery—but the routing pipeline makes their output reviewable.

*Future work.* The audit leaves three concrete follow-ups, each addressing one of the threats catalogued in §7. (i) *Reproducibility:* re-execute each SE-PASS-yielding cell $k = 5$ times to convert the headline point estimate into a confidence interval. (ii) *Construct validity:* run a randomised human re-rating on the 146 main-corpus PARTIAL outputs to quantify the gap between SE-PASS and a relaxed reviewer-acceptable verdict (RA-PASS). (iii) *Calibration extension:* repeat the 96-cell human-rater pilot on a CapBench stratified sample so the panel-vs-human alignment can be reported across the main corpus rather than only the calibration subset. Each closes a known gap that limits how strongly the deployment-policy claims in §5 generalise outside Huawei's repositories.

## Data Availability

The replication package is published as a Zenodo deposit (DOI fixed at camera-ready); a Git mirror is provided alongside the DOI for browsing. The deposit contains all artefacts listed below. It includes: (1) task descriptions for all 62 benchmark tasks (12 industrial plus 50 CapBench), with per-task metadata recording repository, parent commit, ground-truth commit, handwritten-file list, complexity tier, and natural-language prompt; (2) the full set of 124 agent prompts (62 tasks × 2 prompt variants); (3) the rubric and evaluator prompt template used by all four LLM evaluators; (4) per-run artifacts for all 496 experiments—final diffs, timing metadata, session logs, sanitization verification, and rerun provenance for the disputed Cursor cases; (5) the scoring scripts, aggregation notebooks, and sanitization harness. We distribute `reports/eval_scores_v2_long.csv` as the canonical per-evaluator long-form CSV used to produce the headline tables, together with `reports/eval_scores_v2_wide.csv` for run-level cross-checks and `paper2/compute_stats.py` for recomputing the aggregate statistics. The artifact contains no proprietary Huawei source code; for restricted industrial tasks it provides sanitized metadata, prompts, scores, and patch summaries, while public upstream repositories referenced by CapBench tasks can be reconstructed from the recorded SHAs.

## References


[1] Reem Aleithan. 2025. Revisiting SWE-Bench: On the Importance of Data Quality for LLM-Based Code Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 235–236. doi:10.1109/ICSE-Companion66252.2025.00075

[2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. *arXiv preprint arXiv:2410.06992* (2024). doi:10.48550/arXiv.2410.06992

[] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. ACM, New York, NY, USA, 185–196. arXiv:2402.09171 doi:10.1145/3663529.3663839

[3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 doi:10.48550/arXiv.2505.20411

[4] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, A. Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 doi:10.48550/arXiv.2406.01304

[5] Hao Chen, Chengze Li, and Jia Li. 2025. FeatBench: Evaluating Coding Agents on Feature Implementation for Vibe Coding. arXiv:2509.22237 doi:10.48550/arXiv.2509.22237

[6] Jiaju Chen, Yuxuan Lu, Xiaojie Wang, Huimin Zeng, Jing Huang, Jiri Gesi, Ying Xu, Bingsheng Yao, and Dakuo Wang. 2025. Multi-Agent-as-Judge: Aligning LLM-Agent-Based Automated Evaluation with Multi-Dimensional Human Evaluation. arXiv:2507.21028 doi:10.48550/arXiv.2507.21028

[7] Silin Chen, Shaoxin Lin, Xiaodong Gu, Yuling Shi, Heng Lian, Longfei Yun, Dong Chen, Weiguo Sun, Lin Cao, and Qianxiang Wang. 2025. SWE-Exp: Experience-Driven Software Issue Resolution. arXiv:2507.23361 doi:10.48550/arXiv.2507.23361

[8] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. arXiv:2502.01821 doi:10.48550/arXiv.2502.01821

[9] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Mantas Mazeika, and Aleksander Madry. 2024. Introducing SWE-bench Verified. OpenAI research release. https://openai.com/index/introducing-swe-bench-verified/

[10] I. Costa. 2026. AgentSpawn: Adaptive Multi-Agent Collaboration Through Dynamic Spawning for Long-Horizon Code Generation.

[11] Giuseppe Crupi, Rosalia Tufano, Alejandro Velasco, Antonio Mastropaolo, Denys Poshyvanyk, and Gabriele Bavota. 2025. On the Effectiveness of LLM-as-a-Judge for Code Generation and Summarization. *IEEE Transactions on Software Engineering* 51 (2025), 2329–2345. doi:10.1109/TSE.2025.3586082

[12] Xiang Deng, Jeff Da, Edwin Pan, Yan He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 doi:10.48550/arXiv.2509.16941

[13] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, et al. 2025. NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents. arXiv:2512.12730

[14] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33 (2023), 1–38. doi:10.1145/3672459

[15] Ramtin Ehsani, Sakshi Pathak, Shriya Rawal, Abdullah Al Mujahid, M. M. Imran, and Preetha Chatterjee. 2026. Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. arXiv:2601.15195 doi:10.48550/arXiv.2601.15195

[16] Ora Nova Fandina, Eitan Farchi, Shmulik Froimovich, Rami Katan, Alice Podolsky, Orna Raz, and Avi Ziv. 2025. Automated Validation of LLM-based Evaluators for Software Engineering Artifacts. arXiv:2508.02827 doi:10.48550/arXiv.2508.02827

[17] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. 2025. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. arXiv:2507.23370 doi:10.48550/arXiv.2507.23370

[18] GitHub. 2024. Octoverse: The State of Open Source and Rise of AI in 2024. https://github.blog/news-insights/octoverse/octoverse-2024/.

[19] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Yuanzhuo Wang, and Jian Guo. 2024. A Survey on LLM-as-a-Judge. arXiv:2411.15594 doi:10.48550/arXiv.2411.15594

[20] Lianghong Guo, Yanlin Wang, Caihua Li, Pengyu Yang, Jiachi Chen, Wei Tao, Yingtian Zou, Duyu Tang, and Zibin Zheng. 2025. SWE-Factory: Your Automated Factory for Issue Resolution Training Data and Evaluation Benchmarks. arXiv:2506.10954 doi:10.48550/arXiv.2506.10954

[] Jingxuan He, Mark Niklas Müller, Niels Mündler, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 81857–81887. arXiv:2406.12952 doi:10.52202/079017-2601

[21] Junda He, Jieke Shi, Terry Yue Zhuo, Christoph Treude, Jiamou Sun, Zhenchang Xing, Xiaoning Du, and David Lo. 2025. From Code to Courtroom: LLMs as the New Software Judges. arXiv:2503.02246 doi:10.48550/arXiv.2503.02246

[22] Junda He, Jieke Shi, Terry Yue Zhuo, Christoph Treude, Jiamou Sun, Zhenchang Xing, Xiaoning Du, and David Lo. 2025. LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead. arXiv:2510.24367 doi:10.48550/arXiv.2510.24367

[] Smriti Jha, Matteo Paltenghi, Chandra Maddila, Vijayaraghavan Murali, Shubham Ugare, and Satish Chandra. 2026. ProdCodeBench: A Production-Derived Benchmark for Evaluating AI Coding Agents. arXiv:2604.01527 https://arxiv.org/abs/2604.01527

[23] Mohan Jiang, Dayuan Fu, Junhao Shi, Ji Zeng, Weiye Si, Keyu Li, Xuefeng Li, Yang Xiao, Wenjie Li, Dequan Wang, and Pengfei Liu. 2026. daVinci-Agency: Unlocking Long-Horizon Agency Data-Efficiently.

[24] Carlos E. Jimenez, Kilian Lieret, Karthik Narasimhan, Ofir Press, Alexander Wettig, John Yang, and Shunyu Yao. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 50528–50652. arXiv:2405.15793 doi:10.52202/079017-1601

[25] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 52 pages. arXiv:2310.06770 https://arxiv.org/abs/2310.06770

[26] Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. 2025. SWE-Bench-CL: Continual Learning for Coding Agents. arXiv:2507.00014 doi:10.48550/arXiv.2507.00014

[27] Jaehun Jung, Faeze Brahman, and Yejin Choi. 2024. Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement. arXiv:2407.18370 doi:10.48550/arXiv.2407.18370

[28] Muhammad Ahmad Khan, Hasnain Ali, M. Rana, Muhammad Ilyas, and A. A. Bangash. 2026. On the Adoption of AI Coding Agents in Open-source Android and iOS Development.

[29] Caihua Li, Lianghong Guo, Yanlin Wang, Daya Guo, Wei Tao, Zhenyu Shan, Mingwei Liu, Jiachi Chen, Haoyu Song, Duyu Tang, Hongyu Zhang, and Zibin Zheng. 2026. Advances and Frontiers of LLM-based Issue Resolution in Software Engineering: A Comprehensive Survey. arXiv:2601.11655 doi:10.48550/arXiv.2601.11655

[30] Chunyang Li, Yilun Zheng, Xinting Huang, Tianqing Fang, Jiahao Xu, Yangqiu Song, Lihui Chen, and Han Hu. 2025. WebDevJudge: Evaluating (M)LLMs as Critiques for Web Development Quality. arXiv:2510.18560 doi:10.48550/arXiv.2510.18560

[31] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. arXiv:2412.05579

[32] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2025. SWE-Debate: Competitive Multi-Agent Debate for Software Issue Resolution. arXiv:2507.23348 doi:10.48550/arXiv.2507.23348

[33] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2026. AIDev: Studying AI Coding Agents on GitHub. doi:10.1145/3793302.3797249

[34] Yuran Li, J. Mohamud, Chongren Sun, Di Wu, and Benoit Boulet. 2025. Leveraging LLMs as Meta-Judges: A Multi-Agent Framework for Evaluating LLM Judgments. arXiv:2504.17087 doi:10.48550/arXiv.2504.17087

[35] Dario Loi, Elena Maria Muia, F. Siciliano, Giovanni Trappolini, Vincenzo Crisa, Peter Kruger, and Fabrizio Silvestri. 2025. AutoBench: Automating LLM Evaluation through Reciprocal Peer Assessment. arXiv:2510.22593 doi:10.48550/arXiv.2510.22593

[36] Junyi Lu, Xiaojia Li, Zihan Hua, Lei Yu, Shiqi Cheng, Li Yang, Fengjun Zhang, and Chun Zuo. 2024. DeepCRCEval: Revisiting the Evaluation of Code Review Comment Generation. arXiv:2412.18291 doi:10.48550/arXiv.2412.18291

[37] D. Pham and Taher A. Ghaleb. 2026. Code Change Characteristics and Description Alignment: A Comparative Study of Agentic versus Human Pull Requests.

[38] Giovanni Pinna, Jingzhi Gong, David Williams, and Federica Sarro. 2026. Comparing AI Coding Agents: A Task-Stratified Analysis of Pull Request Acceptance.

[39] Thanosan Prathifkumar, N. Mathews, and Mei Nagappan. 2025. Does SWE-Bench-Verified Test Agent Ability or Model Memory?

[40] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub.

[41] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating Agent-Based Program Repair at Google. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Los Alamitos, CA, USA, 365–376. arXiv:2501.07531 doi:10.1109/ICSE-SEIP66354.2025.00038

[42] Linxin Song, Yutong Dai, Viraj Prabhu, Jieyu Zhang, Taiwei Shi, Li Li, Junnan Li, Silvio Savarese, Zeyuan Chen, Ran Xu, and Caiming Xiong. 2025. CoAct-1: Computer-using Agents with Coding as Actions. arXiv:2508.03923 doi:10.48550/arXiv.2508.03923

[] Tao Sun, Jian Xu, Yuanpeng Li, Zhaodong Yan, Ge Zhang, Lintao Xie, Lu Geng, Zheng Wang, Yueyan Chen, Qin Lin, Wenbo Duan, Kaixin Sui, and Yuanshuo Zhu. 2025. BitsAI-CR: Automated Code Review via LLM in Practice. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, New York, NY, USA, 274–285. arXiv:2501.15134 doi:10.1145/3696630.3728552

[43] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2024. Human-In-The-Loop Software Development Agents. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 342–352. doi:10.1109/ICSE-SEIP66354.2025.00036

[44] Minh-Tin Thai, Tue Le, Dung Manh Nguyen, Huy Nhat Phan, and Nghi D. Q. Bui. 2025. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. arXiv:2512.18470 doi:10.48550/arXiv.2512.18470

[45] Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. 2024. Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges. arXiv:2406.12624 doi:10.48550/arXiv.2406.12624

[46] En-Qi Tseng, Pei-Cing Huang, Chan Hsu, Peng-Yi Wu, Chan-Tung Ku, and Yihuang Kang. 2024. CodEv: An Automated Grading Framework Leveraging Large Language Models for Consistent and Constructive Feedback. In *2024 IEEE International Conference on Big Data (BigData)*. 5442–5449. doi:10.1109/BigData62323.2024.10825949

[47] Ngoc Phuoc An Vo, B. Paulovicks, and V. Sheinin. 2025. LLM-as-a-Judge for Reference-less Automatic Code Validation and Refinement for Natural Language to Bash in IT Automation. arXiv:2506.11237 doi:10.48550/arXiv.2506.11237

[48] Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. 2025. SWE-Dev: Building Software Engineering Agents with Training and Inference Scaling. 3742–3761. doi:10.48550/arXiv.2506.07636

[49] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proceedings of the ACM on Software Engineering* 2 (2025), 1955–1977. doi:10.1145/3728963

[50] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. arXiv:2509.14745 doi:10.48550/arXiv.2509.14745

[51] Martin Weyssow, Aton Kamanda, Xin Zhou, and Houari Sahraoui. 2024. CodeUltraFeedback: An LLM-as-a-Judge Dataset for Aligning Large Language Models to Coding Preferences. *ACM Transactions on Software Engineering and Methodology* (2024). doi:10.1145/3736407

[52] Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. 2025. Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases. arXiv:2512.10398 doi:10.48550/arXiv.2512.10398

[53] Yang Wu, Yao Wan, Zhaoyang Chu, Wenting Zhao, Ye Liu, Hongyu Zhang, Xuanhua Shi, Hai Jin, and Philip S. Yu. 2024. Can Large Language Models Serve as Evaluators for Code Summarization? *IEEE Transactions on Software Engineering* 51 (2024), 3205–3217. doi:10.1109/TSE.2025.3595283

[54] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. arXiv:2407.01489 doi:10.1145/3715754

[55] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? arXiv:2511.13646 doi:10.48550/arXiv.2511.13646

[56] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. arXiv:2501.05040 doi:10.48550/arXiv.2501.05040

[57] Kai Xu, YiWei Mao, XinYi Guan, and ZiLong Feng. 2025. Web-Bench: A LLM Code Benchmark Based on Web Standards and Frameworks. arXiv:2505.07473 doi:10.48550/arXiv.2505.07473

[58] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-Based

Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, Kui Liu,,

Framework for CI/CD Failure Detection and Remediation with Industrial Validation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Los Alamitos, CA, USA, 3742–3753. arXiv:2506.03691 doi:10.1109/ASE63991.2025.00310

[59] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and L. Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 doi:10.48550/arXiv.2504.02605

[60] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. 2024. SWE-bench-java: A GitHub Issue Resolving Benchmark for Java. arXiv:2408.14354 doi:10.48550/arXiv.2408.14354

[61] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, J. Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Yudong Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 doi:10.48550/arXiv.2505.23419

[62] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. doi:10.1145/3650212.3680384

[63] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. 2025. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks.

[64] Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Dandan Tu, and Zhaoxiang Zhang. 2026. FeatureBench: Benchmarking Agentic Coding for Complex Feature Development.

[65] Xin Zhou, Kisub Kim, Ting Zhang, Martin Weyssow, Luis F. Gomes, Guang Yang, and David Lo. 2025. An LLM-as-Judge Metric for Bridging the Gap with Human Evaluation in SE Tasks. arXiv:2505.20854 doi:10.48550/arXiv.2505.20854

[66] Qinglin Zhu, Tianyu Chen, Shuai Lu, Lei Ji, Runcong Zhao, Murong Ma, Xiangxiang Dai, Yulan He, Lin Gui, Peng Cheng, and Yeyun Gong. 2026. Pull Requests as a Training Signal for Repo-Level Code Editing.